

Guía de estudio — Presentación del CRUD de Transacciones

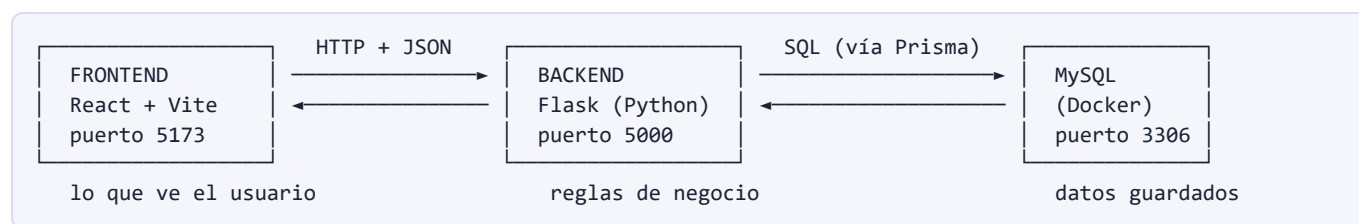
Esta guía va en orden de importancia. Si solo tienes tiempo para una parte, estudia **las secciones 1 a 4**: son las que un profesor casi siempre pregunta. Las demás te dan seguridad para responder preguntas de detalle.

0. La idea en 30 segundos (apréndela de memoria)

"Es una aplicación web para crear, consultar, editar y eliminar transacciones (un CRUD). Tiene dos programas separados: un **backend** en Python con Flask, que expone una API REST y guarda los datos en MySQL usando Prisma, y un **frontend** en React, que muestra la interfaz y se comunica con el backend por HTTP enviando y recibiendo JSON. El backend está organizado en tres capas: rutas, controladores y modelos. Mis mejoras fueron validación centralizada, códigos de error HTTP correctos, un endpoint de resumen y, en la interfaz, tarjetas de resumen, búsqueda, filtros y confirmación al eliminar."

Si puedes decir eso con tus propias palabras y explicar cada término que aparece, ya tienes la mitad de la presentación.

1. La arquitectura general: tres piezas que se comunican



Qué debes entender:

- **Ciente-servidor.** El frontend es el *cliente* (pide cosas) y el backend es el *servidor* (responde). Son **dos procesos distintos**, en puertos distintos. Por eso se encienden con comandos distintos (`npm run dev` y `python app.py`).
- **El frontend nunca toca la base de datos.** Solo habla con el backend. Si te preguntan por qué: por seguridad (las credenciales de la BD solo las conoce el servidor) y porque las reglas de negocio deben aplicarse en un solo lugar.
- **JSON** es el "idioma" común: un texto con formato de diccionario (`{"codigo": "T001", "monto": 50000}`). Python lo convierte en `dict` y JavaScript en un objeto.
- **Docker** solo se usa para tener MySQL corriendo en un contenedor (`docker start empresa`). No hace falta saber mucho más que eso.

- **CORS** (`CORS(app)` en `app.py`): el navegador bloquea por defecto que una página en el puerto 5173 llame a un servidor en el puerto 5000. CORS es el permiso que da el backend para que esa llamada sea aceptada.

Pregunta típica: "¿Por qué separar frontend y backend en vez de hacer todo junto?" Respuesta: cada parte tiene una sola responsabilidad, se pueden cambiar por separado (mañana podría existir una app móvil que use la misma API) y la base de datos queda protegida detrás del servidor.

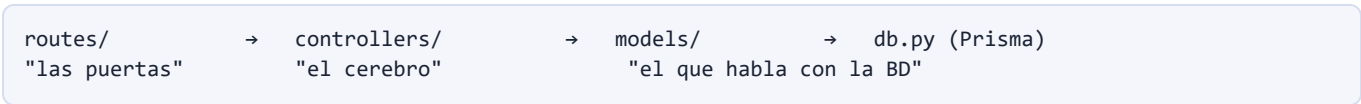
2. El recorrido completo de una petición (lo más importante)

Si entiendes este recorrido, entiendes el proyecto. Practica explicarlo **siguiendo el código con el dedo**. Ejemplo: el usuario crea la transacción `T005`.

Paso	Dónde ocurre	Qué pasa
1	<code>Transacciones.jsx</code> → <code>enviarFormulario</code>	El usuario pulsa "Crear". <code>evento.preventDefault()</code> evita que el navegador recargue la página. Se arma el objeto <code>datos</code> y se convierten <code>monto</code> e <code>impacto</code> a número con <code>Number(...)</code> .
2	<code>api/transacciones.js</code> → <code>crearTransaccion</code> → <code>peticion</code>	Se hace <code>fetch</code> con método <code>POST</code> a <code>http://localhost:5000/api/transacciones/</code> y el objeto convertido a texto con <code>JSON.stringify</code> .
3	<code>app.py</code>	Flask recibe la petición. Como la URL empieza por <code>/api/transacciones</code> , la manda al <i>blueprint</i> de transacciones.
4	<code>routes/transaccion_routes.py</code> → <code>crear</code>	La ruta <code>"/</code> con método <code>POST</code> corresponde a esta función. Saca el JSON con <code>request.get_json(silent=True)</code> y llama al controlador.
5	<code>controllers/transaccion_controller.py</code> → <code>crear_transaccion</code>	Llama a <code>validar_datos</code> . Si hay error → responde 400 . Si el código ya existe → 409 . Si todo está bien, llama al modelo.
6	<code>models/transaccion_model.py</code> → <code>crear</code>	<code>db.transaccion.create(data=datos)</code> : Prisma genera el <code>INSERT</code> de SQL y lo ejecuta en MySQL.
7	De vuelta en el controlador	Devuelve la transacción creada y el código 201 Created . La ruta la convierte a JSON con <code>jsonify</code> .
8	De vuelta en <code>peticion</code>	Si <code>respuesta.ok</code> es falso, lanza un <code>Error</code> con el mensaje del backend. Si salió bien, devuelve los datos.
9	De vuelta en <code>enviarFormulario</code>	Muestra el mensaje de éxito, limpia el formulario y llama a <code>cargarDatos()</code> para volver a pedir la lista y el resumen. React vuelve a dibujar la tabla.

Ejercicio: haz este mismo recorrido para **eliminar** y para **editar**. Fíjate en que editar usa `PUT` `/api/transacciones/<id>` y que el `id` viaja en la URL, no en el cuerpo.

3. Arquitectura en capas del backend



Capa	Responsabilidad	Qué NO hace
routes	Decide qué función atiende cada URL + método HTTP.	No valida ni toca la BD.
controllers	Valida, aplica las reglas del negocio y decide el código HTTP.	No sabe cómo se escribe una consulta.
models	Ejecuta el CRUD contra la base de datos.	No decide si un dato es válido.

Conecta esto con lo que ya sabes de POO:

- **Principio de responsabilidad única** (la "S" de SOLID): cada capa tiene un solo motivo para cambiar. Si cambiamos MySQL por PostgreSQL, solo se toca `models/` (y el `schema.prisma`).
- **Abstracción:** el controlador llama `transaccion_model.crear(datos)` sin saber cómo funciona por dentro. Es lo mismo que llamar un método de un objeto sin conocer su implementación.
- **Encapsulación:** `validar_datos` cumple el mismo papel que un `setter` que protege un atributo: nada entra a la base de datos sin pasar por esa función.

Pregunta probable: "¿Dónde está la POO en este proyecto, si casi todo son funciones?" Prepárate porque es una pregunta muy natural para tu semestre. Una respuesta honesta: - Los archivos se organizan como **módulos** (cada uno agrupa funciones de una sola responsabilidad), que cumplen un papel parecido al de una clase sin estado. - Sí hay objetos: `db = Prisma()` es **una instancia** que todo el backend comparte; cada transacción que devuelve Prisma es un **objeto** de la clase `Transaccion` generada a partir del `schema.prisma`, con atributos (`t.codigo`, `t.monto`) y métodos (`t.dict()`). - Los principios de POO (encapsulación, abstracción, responsabilidad única) se aplican aunque no se escriba la palabra `class`. - Si quisieran más POO, el controlador podría ser una clase `TransaccionService` con métodos. Puedes mencionarlo como mejora posible.

4. HTTP y REST: verbos y códigos de estado

Los verbos y las rutas (el "contrato" de la API)

Operación CRUD	Verbo HTTP	Ruta	Respuesta correcta
Read (todas)	GET	<code>/api/transacciones/</code>	200 + lista
Read (una)	GET	<code>/api/transacciones/5</code>	200, o 404 si no existe

Operación CRUD	Verbo HTTP	Ruta	Respuesta correcta
Resumen	GET	/api/transacciones/resumen	200 + estadísticas
Create	POST	/api/transacciones/	201 Created
Update	PUT	/api/transacciones/5	200
Delete	DELETE	/api/transacciones/5	200

REST significa, en términos sencillos: la URL dice *sobre qué recurso* trabajas (las transacciones, o la transacción 5) y el verbo dice *qué haces* con él.

Códigos de estado que usa tu proyecto

Código	Significado	Cuándo lo devuelve tu backend
200 OK	Todo bien	Listar, obtener, actualizar, eliminar
201 Created	Se creó algo nuevo	Crear transacción
400 Bad Request	El cliente mandó datos mal	Falló <code>validar_datos</code>
404 Not Found	No existe	Id inexistente, o URL mal escrita
405 Method Not Allowed	Ese verbo no existe en esa ruta	Por ejemplo <code>PATCH</code>
409 Conflict	Choca con algo que ya existe	Código de transacción repetido
500 Internal Server Error	Falla del servidor	Excepción inesperada (por ejemplo, BD apagada)

Regla para recordar: **4xx = culpa del cliente, 5xx = culpa del servidor.**

Preguntas probables: - "¿Por qué 409 y no 400 para el código duplicado?" → Los datos tienen el formato correcto (no es 400); el problema es que chocan con un registro existente. 409 dice exactamente eso. - "¿Por qué no se envía `str(e)` al cliente en el error 500?" → El texto interno de la excepción puede revelar detalles de la base de datos (nombres de tablas, conexión). El detalle se imprime en la consola del servidor y al cliente le llega un mensaje genérico. - "¿Por qué `/resumen` no se confunde con `/<int:id>?`" → El convertidor `int` solo acepta números, así que "resumen" nunca coincide con esa ruta.

5. La validación (`validar_datos`): tu mejora principal

Abre `transaccion_controller.py` y asegúrate de poder explicar **cada** `if`. Es lógica de programación pura, tu terreno.

1. `isinstance(datos, dict)` → si el cuerpo no era JSON, `datos` llega como `None`.
2. Lista por comprensión `faltantes = [campo for campo in ... if ...]` → recorre los campos obligatorios y junta los que faltan.
3. `.strip().upper()` → normaliza: `" t9 "` se convierte en `"T9"`.

4. `tipo not in TIPOS_VALIDOS` → solo se aceptan los tres tipos.
5. `int(monto) != monto` → detecta decimales: `2.5` no es igual a `int(2.5) == 2`.
6. `monto <= 0` → una transacción de \$0 no tiene sentido.
7. Se construye un **diccionario nuevo** con solo 4 campos (**lista blanca**): si alguien manda `"id": 99`, se descarta.

Detalles que impresionan si los sabes explicar: - **Devuelve una tupla** (`datos_limpios, error`) y se desempaqueta así: `limpios, error = validar_datos(datos)`. Es una forma de devolver dos resultados. - `_es_numero` **excluye** `bool` porque en Python `True` es un `int` que vale 1. Sin ese cuidado, `monto=True` pasaría como un monto de 1. - **DRY**: antes crear y actualizar repetían la validación; ahora la escriben una sola vez. - **Constantes arriba del archivo** (`TIPOS_VALIDOS`): si aparece un tipo nuevo, se cambia en un solo lugar.

Pregunta trampa: "Si ya validas en el frontend (`required`, `min="1"`), ¿para qué validar en el backend?" → La validación del frontend es solo comodidad para el usuario y se puede saltar (por ejemplo, llamando la API con `curl` o Postman). **La validación que protege los datos es siempre la del backend.**

6. El manejo de errores (`try / except` ↔ `try / catch`)

Es el mismo concepto en los dos lenguajes, y es un buen punto para mostrar que entiendes el flujo:

- **Backend:** el controlador envuelve las operaciones de BD en `try / except`. Si algo falla, responde 500 en lugar de que el servidor se caiga.
- **Frontend:** `peticion()` **lanza** (`throw`) un `Error` cuando la respuesta no es exitosa. La página lo **atrapa** con `try / catch` y muestra `e.message` en rojo.
- **finally** se ejecuta siempre, haya error o no. Por eso `setEnviando(false)` y `setCargando(false)` van ahí: el botón se reactiva en cualquier caso.

Distingue los dos tipos de fallo que maneja `peticion()`: 1. **No hay conexión** (backend apagado) → `fetch` lanza una excepción → "No se pudo conectar con el servidor". 2. **Sí hubo respuesta, pero con error** (400, 404, 409...) → `fetch` NO lanza excepción por sí solo; por eso se revisa `respuesta.ok` a mano.

7. Prisma y la base de datos

- **ORM** (*Object-Relational Mapping*): traduce entre objetos del lenguaje y filas de una tabla. En vez de escribir `SELECT * FROM Transaccion ORDER BY id DESC`, escribes `db.transaccion.find_many(order={"id": "desc"})`.
- `schema.prisma` define la tabla. Es como el diagrama de una clase:
- `id Int @id @default(autoincrement())` → llave primaria que la BD numera sola.
- `codigo String @unique` → no puede haber dos iguales (por eso existe el 409).
- `.env` guarda `DATABASE_URL` (dónde está la base y con qué usuario). Se carga con `load_dotenv()` en `db.py`.
- `prisma db push` crea o ajusta las tablas en MySQL según el esquema y genera el cliente de Python.

- Sobre el modelo `Usuario` : existe en el esquema pero **no se usa todavía**. Dilo tú antes de que te lo pregunten, como trabajo futuro.

8. React: los conceptos que necesitas

No necesitas dominar React, pero sí estos conceptos, porque aparecen en tu código:

Concepto	Dónde aparece	Explicación sencilla
Componente	<code>Transacciones</code> , <code>TarjetasResumen</code> , <code>Tarjeta</code>	Una función que devuelve lo que se ve en pantalla. Parecido a una clase reutilizable de la interfaz.
JSX	Todo el <code>return (...)</code>	HTML escrito dentro de JavaScript. <code>{variable}</code> inserta valores.
Props	<code><TarjetasResumen resumen={resumen} /></code>	Datos que un componente padre le pasa a un hijo, como los parámetros de un constructor.
Estado (useState)	<code>const [transacciones, setTransacciones] = useState([])</code>	Una variable que, al cambiar con su <code>set...</code> , hace que React vuelva a dibujar la pantalla.
useEffect	Cargar datos al iniciar; borrar el mensaje de éxito a los 3 s	Código que se ejecuta <i>después</i> de dibujar. Con <code>[]</code> corre una sola vez; con <code>[exit]</code> corre cada vez que cambia <code>exit</code> .
Formulario controlado	<code>value={formulario.codigo}</code> + <code>onChange={manejarCambio}</code>	El valor del input vive en el estado; React es la "única fuente de verdad".
Estado derivado	<code>transaccionesVisibles = transacciones.filter(...)</code>	La lista filtrada no se guarda: se calcula en cada render a partir del estado. Así nunca queda desactualizada.
Renderizado condicional	<code>{error && <p>...</p>}</code> , <code>editandoId ? ... : ...</code>	Mostrar algo solo si se cumple una condición. Es lógica booleana.
.map() + key	Filas de la tabla, opciones del <code>select</code>	Convierte un arreglo de datos en un arreglo de elementos visuales. <code>key</code> ayuda a React a identificar cada fila.

Tres detalles que conviene saber explicar: - `async / await` : las peticiones tardan, así que `await` espera la respuesta sin congelar la página. - `Promise.all` en `cargarDatos` : pide la lista y el resumen **al mismo tiempo**, no uno después del otro. - `{ ...anterior, [name]: valor }` : copia el objeto del formulario y reemplaza solo el campo que cambió. `[name]` usa el atributo `name` del input como nombre de la propiedad, así una sola función sirve para los cuatro campos.

9. El endpoint de resumen: patrón acumulador

En `obtener_resumen` se recorre la lista **una sola vez** y se van sumando cantidad y monto por tipo en un diccionario. Es el mismo patrón de *contador/acumulador* de lógica de programación, pero guardado en un diccionario en vez de en variables sueltas.

`setdefault` crea el grupo si no existía (por si en la BD hay un tipo antiguo que ya no está en `TIPOS_VALIDOS`).

Pregunta probable: "¿Por qué calcular el resumen en el backend y no en el frontend?" → El frontend podría calcularlo, pero así la regla queda en un solo lugar y cualquier otro cliente de la API recibe el mismo resultado. Como mejora, con muchos datos convendría que lo calculara directamente la base de datos (con `SUM` y `GROUP BY`).

10. Puntos débiles: dílos tú antes de que te los señalen

Reconocer las limitaciones demuestra que entiendes el proyecto. Ya tienes varias en `REPORTE_CAMBIOS.md`, sección 6; estas son las que conviene tener listas:

1. **El `.env` está en el repositorio.** En un proyecto real no se sube; se deja un `.env.example` sin contraseñas.
2. **No hay autenticación.** Cualquiera que conozca la URL puede crear o borrar transacciones. El modelo `Usuario` sería el primer paso.
3. **No hay pruebas automáticas.** `validar_datos` es una función pura (misma entrada, misma salida, sin efectos secundarios), ideal para `pytest`.
4. **Sin paginación.** Se traen todas las filas cada vez.
5. **Condición de carrera en el código duplicado** (solo si el profesor es muy técnico): entre "revisar si existe" y "crear" podría llegar otra petición con el mismo código. No se rompe nada porque `@unique` en la BD lo impide, pero en ese caso raro la respuesta sería 500 en vez de 409.
6. **Pendiente de tu reporte:** probar todos los flujos con la base de datos encendida. **Hazlo antes de presentar** (ver sección 11).

11. Antes de presentar: lista de verificación práctica

Ejecuta la aplicación tú mismo y prueba cada caso. Si algo falla en vivo, sabrás por qué.

- ☐ `docker start empresa` y esperar ~15 s.
- ☐ En `BACKEND`: `python app.py` → debe decir `Running on http://127.0.0.1:5000`.
- ☐ En `FRONTEND`: `npm run dev` → abrir la URL que muestre (normalmente `http://localhost:5173`).
- ☐ Crear una transacción válida → mensaje verde, aparece en la tabla, se actualizan las tarjetas.
- ☐ Crear otra con el **mismo código** → mensaje rojo del 409.
- ☐ Editar una (borde ámbar, fila resaltada) y guardar.
- ☐ Eliminar: aparece "¿Eliminar? Sí / No".

- ☐ Buscar por código y filtrar por tipo.
- ☐ Apagar el backend (Ctrl+C) y recargar → mensaje "No se pudo conectar con el servidor".
- ☐ (Opcional, impresiona) Abrir `http://localhost:5000/api/transacciones/resumen` en el navegador para mostrar el JSON crudo.
- ☐ (Opcional) En las herramientas del navegador (F12 → pestaña *Network/Red*) mostrar la petición `POST`, su cuerpo JSON y el código de respuesta.

Guion sugerido para la demo (5–7 minutos)

1. Qué es y para qué sirve (sección 0).
2. Diagrama de las tres piezas (sección 1).
3. Demo en vivo: crear, error de duplicado, editar, eliminar, filtrar.
4. Mostrar el código siguiendo el recorrido de "crear" (sección 2): ruta → controlador → modelo.
5. Tus mejoras, agrupadas: **robustez** (validación y códigos HTTP), **funcionalidad** (resumen, filtros) y **experiencia de usuario** (confirmación, mensajes, estados de carga).
6. Limitaciones y trabajo futuro (sección 10).

12. Autoevaluación

Intenta responder sin mirar. Si dudas en alguna, vuelve a la sección indicada.

1. ¿Qué diferencia hay entre el frontend y el backend? ¿En qué puerto corre cada uno? (1)
2. ¿Qué es CORS y por qué el proyecto lo necesita? (1)
3. Describe paso a paso qué pasa cuando el usuario pulsa "Eliminar" y luego "Sí". (2)
4. ¿Qué capa del backend cambiarías si se reemplaza MySQL por otra base de datos? (3)
5. ¿Qué código HTTP devuelve el backend al editar un id que no existe? ¿Y al crear un código repetido? (4)
6. ¿Por qué hay que validar en el backend si el formulario ya tiene `required`? (5)
7. ¿Por qué `_es_numero(True)` devuelve `False`? (5)
8. ¿Qué es una lista blanca y qué ataque o error evita aquí? (5)
9. ¿Por qué `fetch` no lanza una excepción cuando el servidor responde 404? ¿Cómo lo resuelve `peticion()`? (6)
10. ¿Qué es un ORM? Da un ejemplo de una línea de Prisma y su equivalente en SQL. (7)
11. ¿Qué diferencia hay entre *props* y *estado* en React? (8)
12. ¿Por qué `transaccionesVisibles` no se guarda con `useState`? (8)
13. ¿Qué hace `useEffect(() => { ... }, [])` con el arreglo vacío? (8)
14. ¿Dónde aplicaste encapsulación, DRY y responsabilidad única? (3, 5)
15. Nombra tres limitaciones del proyecto y cómo las resolverías. (10)